

CSS Grid Items Spanning

Easy Guide with Examples

Contents

1	Introduction to Grid Items Spanning	2
1.1	What is Spanning?	2
1.2	Why Spanning Matters	2
1.3	Basic Concept	2
2	Understanding span Keyword	3
2.1	Basic Syntax	3
2.2	How span Works	3
2.3	span is Position-Agnostic	3
3	Spanning with Line Numbers	4
3.1	Start Line / End Line Syntax	4
3.2	Calculating Spans	4
3.3	Practical Example	4
3.4	span vs Line Numbers: Which to Use?	5
4	Combining span with grid-area	6
4.1	Basic Syntax	6
4.2	Using span in grid-area	6
4.3	Practical Example	6
5	Spanning Rows (Vertical Spanning)	8
5.1	Basic Row Spanning	8
5.2	Example 1: Sidebar Layout	8
5.3	Example 2: Image Gallery with Text	9
6	Spanning Columns (Horizontal Spanning)	10
6.1	Basic Column Spanning	10
6.2	Example 1: Full-Width Header and Footer	10
6.3	Example 2: Blog Timeline	11
7	Spanning Multiple Rows and Columns	12
7.1	2D Spanning	12
7.2	Example 1: Magazine Layout with Featured Article	12
7.3	Example 2: Dashboard with Large Widget	13

8	Spanning with Auto-Placement	14
8.1	How Spanning Affects Auto-Placement	14
8.2	Using grid-auto-flow with Spanning	14
8.3	Practical Example: Gallery with Dense Packing	15
9	Spanning with Named Grid Areas	16
9.1	Basic Named Areas	16
9.2	Example: Complex Layout with Named Areas	16
10	Responsive Spanning	18
10.1	Changing Spans at Different Breakpoints	18
10.2	Example: Responsive Gallery	19
11	Best Practices for Spanning	20
11.1	Do's	20
11.2	Don'ts	20
12	Common Mistakes	21
12.1	Mistake 1: Spanning Beyond Grid Bounds	21
12.2	Mistake 2: Creating Unintended Gaps	21
12.3	Mistake 3: Forgetting Responsive Behavior	22
12.4	Mistake 4: Complex Spanning Logic	22
13	Advanced Spanning Techniques	23
13.1	Spanning with CSS Variables	23
13.2	Spanning with calc()	23
13.3	Spanning with Overlapping Items	24
14	Real-World Examples	25
14.1	E-Commerce Product Grid	25
14.2	News Portal Layout	25
14.3	App Dashboard	26
15	Practice Exercises	28
15.1	Exercise 1: Basic Spanning	28
15.2	Exercise 2: Magazine Layout	28
15.3	Exercise 3: Dashboard	28
15.4	Exercise 4: Responsive Gallery	28
16	Summary	30
16.1	Key Takeaways	30
16.2	Quick Reference	30
16.3	Next Steps	31

Chapter 1

Introduction to Grid Items Spanning

Grid Items Spanning is one of the most powerful features of CSS Grid. It allows a single grid item to stretch across multiple rows or columns, creating complex and dynamic layouts without adding extra HTML elements.

1.1 What is Spanning?

Spanning means making an item occupy more space than a single grid cell. Instead of taking up one cell, a spanning item can take up 2, 3, or more cells in either direction (or both).

1.2 Why Spanning Matters

- **Feature Highlights:** Draw attention to important content with larger spans
- **Layout Flexibility:** Create asymmetrical, interesting layouts
- **Hero Sections:** Build full-width or large hero sections easily
- **No HTML Changes:** Achieve different layouts with CSS only
- **Responsive:** Change spans at different breakpoints

1.3 Basic Concept

In a 3x3 grid, an item can normally occupy 1 cell. With spanning, it can occupy:

- 2 cells horizontally (1 column span + 1 more)
- 3 cells horizontally (entire row)
- 2 cells vertically
- 4 cells total (2x2 square)
- And many more combinations

Chapter 2

Understanding span Keyword

The `span` keyword is the simplest way to create spanning items.

2.1 Basic Syntax

```
grid-column: span 2; /* Span 2 columns */
grid-row: span 2;    /* Span 2 rows */
```

The number after `span` tells CSS how many cells to occupy.

2.2 How span Works

```
/* Examples */
grid-column: span 1; /* Default, takes 1 column */
grid-column: span 2; /* Takes 2 columns */
grid-column: span 3; /* Takes 3 columns */

grid-row: span 1;    /* Takes 1 row */
grid-row: span 2;    /* Takes 2 rows */
grid-row: span 4;    /* Takes 4 rows */
```

2.3 span is Position-Agnostic

When you use `span`, CSS calculates the end position automatically based on current position:

```
.item {
  grid-column: span 3;
}

/* If item starts at column 1: ends at column 4 */
/* If item starts at column 2: ends at column 5 */
```

Chapter 3

Spanning with Line Numbers

Instead of using `span`, you can specify exact start and end lines.

3.1 Start Line / End Line Syntax

```
grid-column: 1 / 4;    /* From line 1 to 4 = 3 columns */
grid-column: 2 / 5;    /* From line 2 to 5 = 3 columns */
```

3.2 Calculating Spans

- Line 1 to 3 = **2** columns ($3 - 1 = 2$)
- Line 1 to 4 = **3** columns ($4 - 1 = 3$)
- Line 2 to 6 = **4** columns ($6 - 2 = 4$)

3.3 Practical Example

```
CSS:
.container {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
}

.item1 {
  grid-column: 1 / 3;  /* Lines 1 to 3 = 2 columns */
}

.item2 {
  grid-column: 1 / 4;  /* Lines 1 to 4 = 3 columns */
}

.item3 {
  grid-column: 3 / 4;  /* Lines 3 to 4 = 1 column */
}
```

3.4 `span` vs Line Numbers: Which to Use?

- Use **`span`** when you know how many cells to occupy
- Use **line numbers** when you know exact positions
- **`span`** is often cleaner for general layouts
- **line numbers** give more precise control

Chapter 4

Combining span with grid-area

The `grid-area` shorthand lets you specify both row and column spanning in one property.

4.1 Basic Syntax

```
grid-area: row-start / col-start / row-end / col-end;
```

4.2 Using span in grid-area

```
/* Span 2 rows and 2 columns starting at current position */
grid-area: span 2 / span 2;

/* Span from specific position */
grid-area: 1 / 1 / span 2 / span 2;

/* Mix of line numbers and span */
grid-area: 1 / 1 / 3 / span 3; /* Row 1-3, Col 1, span 3 */
```

4.3 Practical Example

```
HTML:
<div class="container">
  <div class="hero">Hero Section</div>
  <div class="card">Card 1</div>
  <div class="card">Card 2</div>
</div>

CSS:
.container {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-template-rows: repeat(3, 100px);
  gap: 10px;
}
```



```
.hero {  
  grid-area: 1 / 1 / span 1 / span 4; /* Full width */  
  background: #FF6B6B;  
}  
  
.card:nth-child(2) {  
  grid-area: span 2 / span 2;  
  background: #4ECDC4;  
}  
  
.card:nth-child(3) {  
  grid-area: span 2 / span 2;  
  background: #45B7D1;  
}
```

Chapter 5

Spanning Rows (Vertical Spanning)

5.1 Basic Row Spanning

```
.item {  
  grid-row: span 2; /* Occupy 2 rows */  
}
```

5.2 Example 1: Sidebar Layout

```
HTML:  
<div class="container">  
  <aside>Sidebar</aside>  
  <article>Article 1</article>  
  <article>Article 2</article>  
  <article>Article 3</article>  
</div>  
  
CSS:  
.container {  
  display: grid;  
  grid-template-columns: 250px 1fr;  
  grid-template-rows: auto auto auto;  
  gap: 20px;  
}  
  
aside {  
  grid-row: span 3; /* Spans all 3 rows */  
  background: #f5f5f5;  
  padding: 20px;  
}  
  
article {  
  background: #fff;  
  border: 1px solid #ddd;  
  padding: 20px;  
}
```

5.3 Example 2: Image Gallery with Text

HTML:

```
<div class="gallery">
  <img class="large">
  <img class="small">
  <img class="small">
  <p class="description">Text</p>
</div>
```

CSS:

```
.gallery {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: 150px 150px;
  gap: 10px;
}

.large {
  grid-column: span 2;
  grid-row: span 2;
}

.small {
  grid-column: span 1;
  grid-row: span 1;
}

.description {
  grid-column: 3;
  grid-row: 2;
}
```

Chapter 6

Spanning Columns (Horizontal Spanning)

6.1 Basic Column Spanning

```
.item {  
  grid-column: span 3; /* Occupy 3 columns */  
}
```

6.2 Example 1: Full-Width Header and Footer

```
HTML:  
<div class="container">  
  <header>Header</header>  
  <main>Main</main>  
  <aside>Sidebar</aside>  
  <footer>Footer</footer>  
</div>  
  
CSS:  
.container {  
  display: grid;  
  grid-template-columns: 1fr 250px;  
  gap: 20px;  
}  
  
header {  
  grid-column: span 2; /* Full width */  
}  
  
footer {  
  grid-column: span 2; /* Full width */  
}  
  
main {  
  grid-column: 1;  
}
```

```
aside {  
  grid-column: 2;  
}
```

6.3 Example 2: Blog Timeline

```
HTML:  
<div class="timeline">  
  <article>Post 1</article>  
  <article>Post 2</article>  
  <article>Post 3</article>  
</div>  
  
CSS:  
.timeline {  
  display: grid;  
  grid-template-columns: repeat(2, 1fr);  
  gap: 20px;  
}  
  
article:first-child {  
  grid-column: span 2; /* Full width feature post */  
  background: #FFE66D;  
  padding: 30px;  
  font-size: 1.2em;  
}  
  
article {  
  background: #95E1D3;  
  padding: 20px;  
}
```

Chapter 7

Spanning Multiple Rows and Columns

7.1 2D Spanning

```
.item {  
  grid-column: span 2; /* 2 columns */  
  grid-row: span 2;    /* 2 rows */  
  /* Item now occupies a 2x2 area */  
}
```

7.2 Example 1: Magazine Layout with Featured Article

```
HTML:  
<div class="magazine">  
  <article class="featured">Featured Article</article>  
  <article>Article 1</article>  
  <article>Article 2</article>  
  <article>Article 3</article>  
  <article>Article 4</article>  
</div>  
  
CSS:  
.magazine {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  grid-template-rows: repeat(3, 200px);  
  gap: 20px;  
}  
  
.featured {  
  grid-column: span 2;  
  grid-row: span 2;  
  background: #FF6B6B;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  font-size: 1.5em;  
  color: white;
```

```
}  
  
article {  
  background: #4ECDC4;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  color: white;  
}
```

7.3 Example 2: Dashboard with Large Widget

```
HTML:  
<div class="dashboard">  
  <div class="widget large">Chart</div>  
  <div class="widget">Stat 1</div>  
  <div class="widget">Stat 2</div>  
  <div class="widget">Stat 3</div>  
  <div class="widget">Stat 4</div>  
</div>  
  
CSS:  
.dashboard {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  grid-template-rows: auto auto auto;  
  gap: 15px;  
  padding: 20px;  
}  
  
.large {  
  grid-column: span 2;  
  grid-row: span 2;  
  background: #667eea;  
}  
  
.widget {  
  background: #764ba2;  
  padding: 20px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  color: white;  
}
```

Chapter 8

Spanning with Auto-Placement

8.1 How Spanning Affects Auto-Placement

When items have different spans, CSS Grid’s auto-placement algorithm works harder to find empty cells.

```
.container {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  grid-auto-rows: 100px;  
}  
  
.item1 {  
  grid-column: span 2;  
}  
  
.item2 {  
  grid-column: span 2;  
}  
  
.item3 {  
  grid-column: span 1;  
}  
  
/* Item 3 fills the space left by items 1 and 2 */
```

8.2 Using grid-auto-flow with Spanning

```
.container {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  grid-auto-flow: row dense;  
}  
  
.item1 {  
  grid-column: span 2;  
  grid-row: span 2;  
}
```



```
.item2 {  
  grid-column: span 1;  
  /* 'dense' mode places this in the gap left by item1 */  
}
```

8.3 Practical Example: Gallery with Dense Packing

```
HTML:  
<div class="gallery">  
  <div class="item">1</div>  
  <div class="item">2</div>  
  <div class="item featured">Featured</div>  
  <div class="item">4</div>  
  <div class="item">5</div>  
</div>  
  
CSS:  
.gallery {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  grid-auto-rows: 150px;  
  gap: 10px;  
  grid-auto-flow: row dense;  
}  
  
.featured {  
  grid-column: span 2;  
  grid-row: span 2;  
  background: #FF6B6B;  
}  
  
.item {  
  background: #4ECDC4;  
}
```

Chapter 9

Spanning with Named Grid Areas

9.1 Basic Named Areas

Named grid areas make spanning more readable and maintainable.

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header header"
    "sidebar main main"
    "footer footer footer";
}

header {
  grid-area: header; /* Spans 3 columns automatically */
}

sidebar {
  grid-area: sidebar;
}

main {
  grid-area: main; /* Spans 2 columns */
}

footer {
  grid-area: footer; /* Spans 3 columns */
}
```

9.2 Example: Complex Layout with Named Areas

```
HTML:
<div class="page">
  <header>Header</header>
  <nav>Navigation</nav>
  <main>Main Content</main>
```

```
<aside>Sidebar</aside>
<footer>Footer</footer>
</div>

CSS:
.page {
  display: grid;
  grid-template-columns: 250px 1fr;
  grid-template-rows: auto auto 1fr auto;
  height: 100vh;
  gap: 10px;
  grid-template-areas:
    "header header"
    "nav nav"
    "aside main"
    "footer footer";
}

header {
  grid-area: header;
  background: #333;
  padding: 20px;
}

nav {
  grid-area: nav;
  background: #555;
  padding: 10px;
}

aside {
  grid-area: aside;
  background: #f5f5f5;
}

main {
  grid-area: main;
  background: #fff;
}

footer {
  grid-area: footer;
  background: #333;
}
```

Chapter 10

Responsive Spanning

10.1 Changing Spans at Different Breakpoints

```
/* Mobile: Single column */
.container {
  display: grid;
  grid-template-columns: 1fr;
}

.featured {
  grid-column: span 1;
  grid-row: span 1;
}

/* Tablet: 2 columns */
@media (min-width: 768px) {
  .container {
    grid-template-columns: repeat(2, 1fr);
  }

  .featured {
    grid-column: span 1;
    grid-row: span 1;
  }
}

/* Desktop: 4 columns */
@media (min-width: 1024px) {
  .container {
    grid-template-columns: repeat(4, 1fr);
  }

  .featured {
    grid-column: span 2;
    grid-row: span 2;
  }
}
```

10.2 Example: Responsive Gallery

```
HTML:
<div class="gallery">
  <img>
  <img>
  <img class="featured">
  <img>
</div>

CSS:
.gallery {
  display: grid;
  gap: 10px;
  grid-template-columns: 1fr;
  grid-auto-rows: 200px;
}

@media (min-width: 480px) {
  .gallery {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (min-width: 768px) {
  .gallery {
    grid-template-columns: repeat(3, 1fr);
  }

  .featured {
    grid-column: span 2;
    grid-row: span 2;
  }
}

@media (min-width: 1024px) {
  .gallery {
    grid-template-columns: repeat(4, 1fr);
  }
}
```

Chapter 11

Best Practices for Spanning

11.1 Do's

- **Use spans for emphasis:** Large items draw attention
- **Plan your grid structure:** Sketch layouts before coding
- **Use named areas for readability:** Makes complex layouts clear
- **Keep spans balanced:** Don't make everything span multiple cells
- **Test responsiveness:** Ensure spans work at all breakpoints
- **Use grid-auto-flow dense:** For better space utilization
- **Document your grid:** Comment complex spanning patterns

11.2 Don'ts

- **Don't overuse spanning:** Can make layouts confusing
- **Don't forget negative space:** Let single-cell items breathe
- **Don't create unintended gaps:** Plan your grid carefully
- **Don't hardcode spans:** Use variables or calculations
- **Don't forget accessibility:** Ensure visual order matches DOM order
- **Don't create too many grid lines:** Keep grids simple

Chapter 12

Common Mistakes

12.1 Mistake 1: Spanning Beyond Grid Bounds

```
/* Container has 3 columns (lines 1-4) */
.container {
  grid-template-columns: repeat(3, 1fr);
}

/* WRONG - Trying to span beyond line 4 */
.item {
  grid-column: 2 / 6; /* Line 6 doesn't exist */
  /* Browser will create implicit columns */
}

/* CORRECT */
.item {
  grid-column: 1 / 4; /* Stays within bounds */
}
```

12.2 Mistake 2: Creating Unintended Gaps

```
.container {
  grid-template-columns: repeat(4, 1fr);
}

.item1 {
  grid-column: 1 / 3;
}

.item2 {
  grid-column: span 3; /* Creates gap in column 3 */
}

/* Solution: Plan spans to fill the grid */
.item2 {
  grid-column: 3 / 5; /* Fills columns 3-4 */
}
```

12.3 Mistake 3: Forgetting Responsive Behavior

```
/* WRONG - Same spans on all screens */
.item {
  grid-column: span 3; /* Too large on mobile */
}

/* CORRECT */
.item {
  grid-column: span 1;
}

@media (min-width: 768px) {
  .item {
    grid-column: span 3;
  }
}
```

12.4 Mistake 4: Complex Spanning Logic

```
/* WRONG - Hard to understand */
.item {
  grid-column: calc(var(--cols) + 1) / span 2;
}

/* CORRECT - Clear intention */
.item {
  grid-column: span 2;
  /* Or use named areas */
}
```


Chapter 13

Advanced Spanning Techniques

13.1 Spanning with CSS Variables

```
:root {
  --hero-col-span: 2;
  --hero-row-span: 2;
}

.container {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-template-rows: repeat(3, 150px);
}

.hero {
  grid-column: span var(--hero-col-span);
  grid-row: span var(--hero-row-span);
}

@media (max-width: 768px) {
  :root {
    --hero-col-span: 1;
    --hero-row-span: 1;
  }
}
```

13.2 Spanning with calc()

```
.container {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  --item-span: 2;
}

.item {
  grid-column: span var(--item-span);
}
```

```
.item.large {  
  --item-span: calc(var(--item-span) * 2);  
}
```

13.3 Spanning with Overlapping Items

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 100px);  
  grid-template-rows: repeat(3, 100px);  
  position: relative;  
}  
  
.background {  
  grid-column: span 3;  
  grid-row: span 3;  
  background: url('bg.jpg');  
}  
  
.overlay {  
  grid-column: 1 / 3;  
  grid-row: 1 / 2;  
  background: rgba(0, 0, 0, 0.5);  
  z-index: 1;  
}
```

Chapter 14

Real-World Examples

14.1 E-Commerce Product Grid

```
HTML:
<div class="products">
  <div class="product featured">Featured Product</div>
  <div class="product">Product 1</div>
  <div class="product">Product 2</div>
  <!-- More products -->
</div>

CSS:
.products {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 20px;
}

.featured {
  grid-column: span 2;
  grid-row: span 2;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}

.product {
  background: #f5f5f5;
  padding: 20px;
  text-align: center;
}
```

14.2 News Portal Layout

```
HTML:
<div class="news">
  <article class="hero">Featured Story</article>
  <article>News 1</article>
  <article>News 2</article>
```

```
<article>News 3</article>
</div>

CSS:
.news {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 15px;
}

.hero {
  grid-column: 1 / 3;
  grid-row: 1 / 3;
  background: #FF6B6B;
  padding: 30px;
}

article {
  background: #95E1D3;
  padding: 20px;
}
```

14.3 App Dashboard

```
HTML:
<div class="dashboard">
  <div class="widget chart">Chart</div>
  <div class="widget stat">Stat 1</div>
  <div class="widget stat">Stat 2</div>
  <div class="widget stat">Stat 3</div>
</div>

CSS:
.dashboard {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 200px;
  gap: 20px;
}

.chart {
  grid-column: span 2;
  grid-row: span 2;
  background: #667eea;
}

.stat {
  background: #764ba2;
}

.widget {
  padding: 20px;
}
```

```
color: white;  
display: flex;  
align-items: center;  
justify-content: center;  
}
```

Chapter 15

Practice Exercises

15.1 Exercise 1: Basic Spanning

Create a 4-column grid where:

- First item spans 2 columns
- Second item spans 1 column
- Third item spans 1 column
- Fourth item spans 2 columns and 2 rows

15.2 Exercise 2: Magazine Layout

Build a magazine-style layout with:

- One featured article spanning 2x2
- Six smaller articles in a grid
- Responsive adjustments for mobile

15.3 Exercise 3: Dashboard

Create a dashboard with:

- One large chart spanning 2x2
- Six statistics cards
- Use named grid areas

15.4 Exercise 4: Responsive Gallery

Build a gallery that:

- Shows 1 column on mobile

- Shows 2 columns on tablet with featured item spanning 2x2
- Shows 4 columns on desktop with featured spanning 2x2

Chapter 16

Summary

16.1 Key Takeaways

- **span keyword** makes items occupy multiple cells
- **span works in any direction:** columns, rows, or both
- **Line numbers** provide precise control over spans
- **grid-area** combines row and column spanning
- **Named areas** make complex layouts more readable
- **Responsive spanning** changes at different breakpoints
- **Auto-placement** works intelligently with varied spans
- **Planning** is crucial for avoiding gaps and conflicts

16.2 Quick Reference

```
/* Basic spanning */
grid-column: span 2;      /* Span 2 columns */
grid-row: span 2;         /* Span 2 rows */

/* Line-based spanning */
grid-column: 1 / 4;       /* Line 1 to 4 = 3 columns */
grid-row: 1 / 3;          /* Line 1 to 3 = 2 rows */

/* Combined with grid-area */
grid-area: 1 / 1 / span 2 / span 2;

/* Named areas */
grid-template-areas:
  "header header header"
  "nav main sidebar";

/* Responsive */
@media (min-width: 768px) {
```



```
.item { grid-column: span 2; }  
}
```

16.3 Next Steps

Explore related Grid topics:

- CSS Grid Items Positioning - Fine control over exact placement
- CSS Grid Lines Naming - Named lines for clearer code
- CSS Grid Area Naming - Template-based layouts
- Responsive Grid Techniques - Multi-breakpoint strategies